

- a) The HungryMover class is used to create a thread which moves horizontally across the screen using. It is almost the same as the WordMover class but instead of incrementing the y co-ordinate, we are incrementing the x co-ordinate in order to make the word move horizontally. The HungryMover uses a HungryWord object instead of a FallingWord object. Just like the WordMover class the HungryMover class also has a pause and done atomic Boolean which detects if the game is paused or a condition has been met in order to complete the game.
- b) I have created a HungryWord class in order to control the properties of the HungryWord object which moves horizontally across the game panel. The HungryWord class is similar to the FallingWord class but with some additional changes and methods.

The HungryWord class has a method called “slideAcross” which increments the x value (unlike the y-value for the FallingWord class) in order to move the word horizontally across the panel

```
public synchronized void slideAcross(int slide)
{
    setX((x + slide));
    //System.out.println("Sliding in method || x
value of " + getWord() + "is " + this.getX());
}
```

The HungryWord class also checks whether the horizontally moving word has reached the end of the game panel after every incrementation (see setX () method).

```
public synchronized void setX(int x)
{
    if (x > maxX)
    {
        x = this.maxX;
        end = true;
    }
    this.x = x;
```

The HungryWord class has an additional method called “touched” which check if a HungryWord object is touched by a FallingWord object. It does this by creating a “hi” and ‘lo” variables which sets the boundaries of HungryWord object. “touched’ then calculates the difference between the x and y values of the FallingWord and HungryWord. This difference represents the x and y distance between the HungryWord object and the

FallingWord object. If the distance between these 2 objects are between the boundaries set by the hi and lo variables, the method will return true, else it will return false.

```
public boolean touched (FallingWord fw)
{
    int low = -50 ;
    int hi = 50;
    int xDiff = this.getX() - fw.getX();
    int yDiff = this.getY() - fw.getY();

    System.out.println(fw.getWord() + " xDiff = " + xDiff + "
and yDiff = " + yDiff);

    if (xDiff > low && xDiff < hi && yDiff > low && yDiff < hi)
    {
        //System.out.println(fallingWord.getWord() + "
TOUCHED");
        fw.resetWord();
        return true;
    }
    else
        return false;
}
```

The HungryWord class has a caught method is used in the Caught Word class in order to reset the HungryWord object once it has been successfully typed

```
public synchronized void caught()
{
    setPosition(0, 0);
}
```

note that the setPosition() method makes use of the setX() method stated before

- c) Some changes I made to the existing classes where that I added a HungryWord instance variable to the CatchWord, GamePanel, WordMover and typingTutor classes. In these classes I also modified the constructors to instantiate a HungryWord object in that class.

The HungryWord reference in the CatchWord class was to handle the action if the HungryWord was typed. If the HungryWord was typed, set the HungryWord position to (0, 0) so it would not be on the screen and incremented the score counter to the length of the HungryWord. The catchword class also contained the algorithm which make the lowest word disappear when typed.

```

public void run() {
    int i=0;
    while (i<noWords) {
        while(pause.get()) {};
        if (words[i].matchWord(target)) {
            int index = i;
            int yMax = words [i].getY();
            for (int j = 0 ; j < words.length; j++)
            {
                if (words[j].matches(target) &&
words[j].getY() >= yMax)
                {
                    yMax = words[j].getY();
                    index = j;
                }
            }

            words[index].matchWord(target);
            System.out.println( " score! " + target);
//for checking
            score.caughtWord(target.length());
            //FallingWord.increaseSpeed();
            break;
        }

        if (hWord.matchWord(target))
        {
            System.out.println("HungryWord reset");
            score.caughtWord(target.length());
            hWord.caught();
        }
    }
}

```

The HungryWord in the GamePannel class was to draw the HungryWord string to the game panel. This class also handled the color of the HungryWord string that was printed to the screen. In this class I also added a getValidXPos() to determine all the valid x co-ordinates that the HungryWord could move to.

```

public int getValidXpos() {
    int width = getWidth()-borderWidth*4;
    int x= (int)(Math.random() * width);
    return x;
}

```

In the WordMover class, a reference to the HungryWord class was made in order to determine whether the one of the FallingWords come into contact with the HungryWord. I used the touched() method from the HungryWord class to determine whether these 2 words come into contact. X

```
while (!myWord.dropped() && !done.get())
{
    if (!hword.touched(myWord))
    {
        myWord.drop(10);
        System.out.println(myWord.getWord() + " NOT
TOUCHED and incrementned. myWord co-ordinates = (" +
        myWord.getX() + ", " + myWord.getY() +
        ") and hWord co-ords = (" + hword.getX() + ", " +
        hword.getY() + ")" );
        try {
            sleep(myWord.getSpeed());
        } catch (InterruptedException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }
        while (pause.get() && !done.get()) {
        }
        ;
    }
    else
    {
        score.missedWord();
        myWord.resetWord();
    }
}
```

The reference in the typingTutor class was to create a HungryWord thread which moved horizontally across the screen using the HungryMover class. It was made in the createWordThreads() method which meant that the HungryWord would appear at the same time as the FallingWords and move across the screen

```
public static void createWordMoverThreads() {
    score.reset();
    //System.out.println("Create thread word " +
    hw.getWord());
    hm = new HungryMover(hw, dict, score, startLatch,
    done, pause);
}
```

```

        //initialize shared array of current words with
the words for this game
        for (int i=0;i<noWords;i++) {
            words[i]=new
FallingWord(dict.getNewWord(),gameWindow.getValidXpos
(),yLimit);
        }
        //create threads to move them
        for (int i=0;i<noWords;i++) {
            wrdShft[i] = new WordMover(words[i], hw,
dict,score,startLatch,done,pause);
        }
        hm.start();
        //word movers waiting on starting line
        for (int i=0;i<noWords;i++) {
            wrdShft[i] .start();
        }
    }
}

```

To ensure thread safety I ensured that all the get and set methods in the HungryWord and HungryMover classes were synchronized to only allow one thread access to the method at a time. I ensured Booleans such as done and pause were atomic in order. These atomic variable ensure that another thread cannot interrupt an operation being done by another thread to the variable.